ASSIGNMENT 4 – CO4

NAME : RAJA RAGAVI R K

REGISTRATION : 1911260241

SUBJECT : PYTHON PROGRAMMING

COURSE CODE : CSA0822

GITHUB LINK: https://github.com/RajaRagavi16/CSA0822-1911260241-

TASK 1: FILE-PROCESSING DESIGN REVIEW

AIM:

Diagnose the hostel mess-bill script and repair it with safe streaming, exact decimal amounts and rejected-line reporting.

THEORY:

The original hard-codes a path, leaves the file open, reads all data into memory, hides every failure with bare except/pass, rejects paise with int(), repeatedly copies report text, can process the trailing blank row, and loses rejected records. The repair accepts a path parameter, uses with and line iteration, catches only expected parse errors, uses Decimal, and returns a rejection list. For 50,000 lines it is O(n) time and O(one line plus report) working memory; reading all input raises memory to O(file size). The defense is under 100 words.

PROGRAM CODE:

```
from pathlib import Path
from decimal import Decimal, InvalidOperation
def total_mess(path):
 report=[]; rejected=[]; total=Decimal('0.00')
 with open(path,encoding='utf-8') as f:
  for number,line in enumerate(f,1):
   if not line.strip(): continue
   try:
    reg,name,amount=line.rstrip('\n').split(',')
    value=Decimal(amount)
   except (ValueError,InvalidOperation) as e:
    rejected.append((number,line.rstrip(),str(e))); continue
   total+=value;report.append(f'{name} {value:.2f}')
 return report,total,rejected
p=Path('mess.csv');p.write_text('001,Asha,1250.50\n002,Ravi,900\n003,Bad,xx\n',encoding='utf-8')
report,total,rejected=total_mess(p)
print('report:',report,'total:',total,'rejected:',rejected)
p.unlink()
# Defects repaired: hard-coded path; unclosed file; whole-file read;
# bare except/pass; float/int paise; repeated string +; blank trailing row;
# rejected records not reported. Runtime O(n), streaming memory O(one line).
```

OUTPUT:

```
report: ['Asha 1250.50', 'Ravi 900.00'] total: 2150.50 rejected: [(3, '003,Bad,xx', "[<class
  'decimal.ConversionSyntax'>]")]
```

TASK 2: FROM REQUIREMENT TO FILE-PROCESSING DESIGN

AIM:

Process daily biometric attendance files, calculate monthly percentages, list students below 75%, and log malformed records.

THEORY:

Input rows are register,date,status with P or A. Use dictionaries for present and total counts; write percentages.csv and errors.csv. List files with pathlib, open as UTF-8 in r mode, iterate line by line, and use w mode for new reports. Pseudocode: for each sorted daily file, enumerate lines; validate three fields and status; record filename and line on errors; otherwise update counts; calculate percentages; write both reports. Trace: two valid rows produce 001 at 100%, 002 at 0%; a malformed third row is logged as day1.csv line 3. Reading every file once costs O(f*l); rescanning per student costs O(s*f*l).

PROGRAM CODE:

```
from pathlib import Path
import tempfile

def process_month(folder,outdir):
 present={}; total={}; errors=[]
 for path in sorted(Path(folder).glob('*.csv')):
  with path.open(encoding='utf-8') as f:
   for line_no,line in enumerate(f,1):
    try:
     reg,date,status=line.strip().split(',')
     if status not in ('P','A'): raise ValueError('status must be P or A')
    except ValueError as e: errors.append((path.name,line_no,str(e))); continue
    total[reg]=total.get(reg,0)+1;present[reg]=present.get(reg,0)+(status=='P')
 pct={r:100*present.get(r,0)/n for r,n in total.items()}
 with (Path(outdir)/'percentages.csv').open('w',encoding='utf-8') as f:
  f.write('register,percent\n');f.writelines(f'{r},{v:.1f}\n' for r,v in pct.items())
 with (Path(outdir)/'errors.csv').open('w',encoding='utf-8') as f:
  f.write('file,line,error\n');f.writelines(f'{a},{b},{c}\n' for a,b,c in errors)
 return pct,{r:v for r,v in pct.items() if v<75},errors
with tempfile.TemporaryDirectory(dir=r'C:/Users/RAJA
  RAGAVI/Documents/Codex/2026-11-12/the-first-pdf-contains-my-assisgnment/work') as d:
 root=Path(d);(root/'day1.csv').write_text('001,2026-10-01,P\n002,2026-10-01,A\nbad,line\n',encoding='utf-8');(root/'o
  ut').mkdir()
 print(process_month(root,root/'out'))
 print((root/'out'/'errors.csv').read_text().strip())
```

OUTPUT:

```
({'001': 100.0, '002': 0.0}, {'002': 0.0}, [('day1.csv', 3, 'not enough values to unpack (expected 3, got 2)')])
file,line,error
day1.csv,3,not enough values to unpack (expected 3, got 2)
```

TASK 3: DESIGN THE ERROR HANDLING

AIM:

Identify attendance-file failures, define InvalidRecord, use specific exception handling, and preserve line numbers for damaged rows.

THEORY:

Failure map: missing folder/file raises FileNotFoundError; denied access raises PermissionError; invalid UTF-8 raises UnicodeDecodeError; missing fields can raise IndexError and invalid values can raise ValueError. Raise InvalidRecord at parse_line and catch it per row; catch OSError at the file boundary and use raise ... from e when adding context. The complete try design uses except for specific failures, else for successful processing, and finally for cleanup (with already closes the file). Replacing handlers with except Exception: pass hides programming errors such as a misspelled variable. Validation is O(1) per line; rare exceptions keep the loop efficient.

PROGRAM CODE:

```
import os, tempfile
class InvalidRecord(Exception): pass
def parse_line(line):
 parts=line.strip().split(',')
 if len(parts)!=3: raise InvalidRecord(f'expected 3 fields, found {len(parts)}')
 if parts[2] not in ('P','A'): raise InvalidRecord(f"invalid status {parts[2]!r}")
 return tuple(parts)
def open_attendance(path):
 try:return open(path,encoding='utf-8')
 except OSError as e:raise FileNotFoundError(f'cannot open {path}') from e
def summarise(path):
 present=absent=0;errors=[]
 try:
  with open_attendance(path) as f:
   for number,line in enumerate(f,1):
    try: _,_,status=parse_line(line)
    except InvalidRecord as e: errors.append((number,str(e)));continue
    else:
     if status=='P':present+=1
     else:absent+=1
 finally: pass
 return present,absent,errors
text='192311001,2026-10-08,P\n192311002,2026-10-08,A\n192311003,2026-10-08\n192311004,2026-10-08,L\n'
with tempfile.NamedTemporaryFile('w',delete=False,encoding='utf-8',dir=r'C:/Users/RAJA
  RAGAVI/Documents/Codex/2026-11-12/the-first-pdf-contains-my-assisgnment/work') as f:f.write(text);name=f.name
print(summarise(name));os.remove(name)
try:summarise(name)
except FileNotFoundError as e:print('missing:',type(e).__name__,'cause:',type(e.__cause__).__name__)
# Wrong encoding: UnicodeDecodeError; denied: PermissionError; absent: FileNotFoundError.
# A broad except Exception: pass hides a misspelled variable as well as bad input.
```

OUTPUT:

```
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
missing: FileNotFoundError cause: FileNotFoundError
```

TASK 4: FILE AND EXCEPTION FORENSICS

AIM:

Keep a week of library returns in the log and save an error record when a day’s processing fails.

THEORY:

Opening the log in w mode truncates earlier days; use append mode a so each day is retained. Validate each return record, catch specific file and data errors, and append the date, record and exception to a separate UTF-8 error log. Use with so files close even when processing fails. For n records the work is O(n); memory stays O(one record). This repair addresses both failures described in the task scenario: only the last day remained, and an error message was not saved.

PROGRAM CODE:

```
from pathlib import Path
from datetime import date

def record_returns(log_path, error_path, rows):
 with open(log_path,'a',encoding='utf-8') as log:
  for book,day in rows:
   try:
    if not book: raise ValueError('book id is empty')
    log.write(f'{day},{book}\n')
   except (OSError,ValueError) as e:
    with open(error_path,'a',encoding='utf-8') as errors:
     errors.write(f'{day},{book!r},{type(e).__name__}:{e}\n')

p=Path('returns.log');e=Path('returns_errors.log')
p.write_text('2026-10-01,B001\n',encoding='utf-8')
record_returns(p,e,[('B002','2026-10-02'),('','2026-10-03')])
print('log:',p.read_text().strip().splitlines())
print('errors:',e.read_text().strip())
p.unlink();e.unlink()
```

OUTPUT:

```
log: ['2026-10-01,B001', '2026-10-02,B002']
errors: 2026-10-03,'',ValueError:book id is empty
```